

论文模板 07 · 架构评估与质量属性

配套素材见 [00-项目素材库.md](#)。本主题**理论性最强**，也是最容易写出「架构味」的一类题。⚠ 本主题要求掌握 ATAM / 质量属性场景等**教材概念**，请先复习教材对应章节。

一、真题常见问法

变体 A（架构评估）

请论述你如何对系统架构进行评估，包括评估方法、评估过程与评估结果的应用。

变体 B（质量属性）

请论述质量属性在架构设计中的作用，并结合项目说明你如何实现 性能、可用性、安全性等质量属性之间的权衡。

变体 C（架构决策）

请论述你在架构设计过程中如何进行技术决策，如何处理多个质量属性之间的冲突。

判断：变体 B 是 BASE HOME 的最优命中区——项目有大量真实的「权衡决策」可写（混合存储的取舍、Serverless 的一致性取舍、双 SDK 并存的取舍）。

二、必背理论要点（先背熟，再谈项目）

2.1 六大质量属性

属性	含义	本项目对应设计
性能	响应时间、吞吐量	gRPC (HTTP/2)、读写分离、 <code>fetchSize</code> 控制、DynamoDB 单分区查询
可用性	系统正常运行时间比例	故障域隔离、会话外置、乐观锁重试、幂等
安全性	抵御攻击、保护数据	OAuth2/OIDC、默认拒绝授权、TLS、STS 临时凭证
可修改性	变更的成本与难度	契约单一来源、版本统一治理、模块化拆分
可测试性	发现缺陷的难易	TestContainers、LocalStack、pytest+moto
易用性	用户完成任务的效率	MyPage 界面、ToDo 管理、通知推送

2.2 质量属性场景六要素



示例（本项目可用）：

当营业本部操作员（刺激源）在长假前预约高峰期（环境）提交预约查询请求（刺激）时，Web 与 Service 模块（制品）应在 2 秒内（响应度量）返回结果（响应）。

2.3 ATAM 四个阶段与关键概念

ATAM 四阶段：

- 1. 场景和需求收集 —— 与利益相关者共同梳理质量属性场景
- 2. 架构视图和场景实现 —— 描述架构如何支持各场景
- 3. 属性模型构造和分析 —— 建立性能/可用性等分析模型
- 4. 架构决策与折中 —— 识别权衡点并形成决策

四个关键概念（必考，务必准确使用）：

概念	定义	本项目示例
敏感点	为实现某质量属性，一个或多个构件所具有的关键性质	DynamoDB 的 <code>travel_no</code> 分区键设计是性能的敏感点（决定查询是否落单分区）
权衡点	影响多个质量属性、且对多个属性敏感的敏感点	混合存储选型：提升性能与可扩展性（性能/可修改性），但增加运维复杂度（可维护性/可测试性）
风险点	可能导致问题的架构决策	采用动态版本区间（ <code>latest.release</code> ）——导致构建不可复现（已发生实际故障）
非风险点	经分析确认安全的决策	使用 Spring Session + Redis Cluster 实现会话外置——已有成熟实践支撑

💡 考场上「敏感点/权衡点/风险点」这三个词只要用对，就能明显区别于普通考生。建议在「难点」段中显式使用这些术语。

2.4 常见质量属性策略（战术）

属性	常见策略
可用性	冗余、故障检测、恢复、故障隔离
性能	资源调度、并发、减少计算/存储开销、缓存
可修改性	限制依赖、抽象、信息隐藏、延迟绑定
安全性	认证、授权、限制访问、审计

属性	常见策略
可测试性	记录回放、接口分离、 可观测

三、正文五段框架

第 1 段 • 项目背景与质量属性需求 (约 400 字)

BASE HOME 是 H.I.S. 面向旅游业务的支撑平台，采用 Java 21 + Spring Boot 3.5.7，部署于 AWS ap-northeast-1。本人担任架构师，负责架构设计与质量属性权衡。

在项目初期，我与业务方、运维方共同梳理了关键质量属性需求，并 **将其转化为可度量的质量属性场景**，作为后续所有架构决策的依据：

质量属性	场景 (简化)	度量目标
可用性	长假预约高峰期间，Web 层单实例故障	◆ 服务不中断，用户会话不丢失
性能	高峰期操作员提交预约查询	◆ $P99 \leq 2$ 秒
安全性	未授权用户尝试访问客户消息接口	拒绝访问并记录审计日志
可修改性	新增一类通知渠道	◆ 不改动核心服务，仅新增一个单元
可测试性	开发人员本地验证数据同步逻辑	无需连接真实 AWS 即可完成

这种「场景化」的表达方式，使质量属性从抽象口号变成了可验证的工程目标。

⚠ **注意：**表格中的 ◆ 数字必须由你填真实值。但即使填了估计值，「把需求写成场景」这一动作本身就体现架构思维。

第 2 段 • 评估方法与过程 (约 800 字，本文核心)

推荐用 ATAM 四阶段组织：

阶段一：场景和需求收集

我组织了由业务方（营业本部代表）、运维方、开发负责人参与的评估会议，采用 **效用树 (Utility Tree)** 对质量属性需求进行优先级排序。排序依据为「重要性」与「实现难度」两个维度，高重要性且高难度的需求优先投入架构资源。经排序后，**性能与可用性**被列为最高优先级，**可修改性**次之（因业务政策变动频繁，接口与字段需持续演进）。

阶段二：架构视图和场景实现

我以视图为单位描述架构对场景的支撑，本项目主要涉及四个视图：- **逻辑视图**：8 个 Gradle 模块的职责划分与依赖方向 - **开发视图**：`common`（实体/常量/消息）、`spec`（契约）、`versions`（版本治理）等支撑模块的组织 - **进程视图**：Web (8080)、Service (gRPC 6565)、Batch (无 Web 容器)、FaaS (14 个 Lambda) 的运行时交互 - **部署视图**：AWS ap-northeast-1，多实例部署 + Redis Cluster + Aurora + DynamoDB + S3 + SQS/SNS

阶段三：属性模型构造和分析

针对性能与可用性，我们构造了简化的分析模型：- **性能**：分析关键路径（Web → gRPC → DynamoDB），识别出 DynamoDB 查询是否落在单一分区是主导因素，因此把分区键设计作为性能敏感点重点评审 - **可用性**：分析方法是部署单元逐个标记为「失效」，观察影响范围。分析中发现 Batch 与 Web 共享数据源时，批处理峰值会传导为在线接口延迟——这成为后续读写分离的直接依据

阶段四：架构决策与折中

评估的核心产出是**识别权衡点并形成决策记录**。本项目的三个主要权衡：

权衡一：混合存储 (MySQL + DynamoDB + S3) - 正向：性能↑ (DynamoDB 低延迟、单分区查询)、可修改性↑ (schema-less, 字段演进免 DDL) - 负向：可维护性↓ (多套存储的备份/容量规划)、一致性↓ (需事件驱动达成最终一致) - **决策**：接受。理由是四类数据的访问特征差异足够大，统一存储的代价更高。

权衡二：Serverless 数据同步 - 正向：性能↑ (主链路解耦)、成本↓ (按需计费)、可用性↑ (故障域隔离) - 负向：一致性↓ (最终一致)、可测试性↓ (本地调试困难) - **决策**：接受。理由是同步场景对延迟不敏感，且用 moto/LocalStack 缓解了测试困难。

权衡三：契约单一来源 - 正向：可修改性↑ (接口变更一处生效)、可测试性↑ (构建期暴露不一致) - 负向：构建复杂度↑ (需维护 Docker 化 protoc 环境与三套发布流程) - **决策**：接受。理由见难点素材 ①。

第 3 段 • 敏感点、风险点识别 (约 700 字)

这一节是本文的加分核心，务必显式使用术语。

敏感点识别：

通过评估，我们识别出以下关键敏感点：

敏感点	影响的质量属性	说明
DynamoDB 分区键设计 (<code>travel_no</code> + <code>data_type</code>)	性能	决定查询能否落单分区，是延迟的主导因素
<code>mybatis.defaultFetchSize = 200</code>	性能、可用性	批量游标大小直接关系内存占用与 OOM 风险
Session 存储位置 (本地 vs Redis)	可用性	决定 Web 实例能否水平扩容与无损重启
依赖版本声明方式 (区间 vs 固定)	可修改性、可测试性	决定构建是否可复现
授权校验位置 (接口层 vs 业务代码)	安全性	决定权限遗漏风险的高低

风险点与非风险点：

风险点：采用 `[3,4)` 与 `latest.release` 这样的动态版本区间，使构建产物不可复现，任何一次依赖发布都可能引入破坏性变更。该风险在评估中被识别为高风险——**并且后续真实发生了**（`proto-google-common-protos 2.64.0-rc1` 导致服务启动失败）。这印证了评估的价值：被识别的风险可以提前预案。

非风险点：Spring Session + Redis Cluster 的会话外置方案。该方案在业界有大量成熟实践，且 Redis Cluster 本身提供分片与故障转移，评估确认无需额外预案。

第 4 段 • 难点与解决 (约 700 字)

架构评估主题最佳组合：

优先级	难点	适配理由
首选	② protobuf 版本冲突 (风险点转化为实际故障)	对应「风险点识别」
次选	⑤ 混合存储 (典型权衡点)	对应「权衡点分析」
三选	⑥ Serverless 同步 (一致性与性能的权衡)	对应「属性冲突处理」
加分	③ AWS SDK v1/v2 并存	对应「可修改性的长期成本」

难点写法提示：本主题的难点要用质量属性的语言写，而不是用技术语言。例如不要写「我们把版本锁住了」，而要写「我们识别出该决策对可修改性与可测试性构成风险，通过将隐式版本契约显式化，把风险转化为可控状态」。

第 5 段 • 总结与不足 (约 200 字)

通过本次评估，我们共识别出【◆ 数量】个敏感点、【◆ 数量】个权衡点与【◆ 数量】个风险点，其中【◆ 数量】个风险点在建设期真实发生并有预案应对。系统上线后【◆ 核心质量指标】达到预期。

不足之处：

一是评估主要依靠会议评审与经验判断，尚未引入量化的属性分析模型（如排队模型、可靠性框图）；

二是评估结论缺少工具化沉淀，决策记录以文档形式保存，检索与追溯不够便捷。后续计划引入量化分析工具与架构决策记录（ADR）机制。

通过本项目我深刻体会到，架构评估的价值不在于证明架构正确，而在于**在投入大量资源之前，把"可能的错误"找出来**。

四、本项目填充点速查 (术语对照表)

术语	本项目实例
性能敏感点	DynamoDB 分区键设计； <code>fetchSize=200</code> ； <code>scan-limit=10000</code>
可用性敏感点	Session 存储位置；批量作业数据源隔离；Lambda 函数粒度
安全性敏感点	授权校验位置；凭证获取方式（STS vs 长期密钥）
可修改性敏感点	契约定义位置；依赖版本声明方式；模块依赖方向
可测试性敏感点	云依赖是否可本地模拟（LocalStack / TestContainers / moto）
权衡点	混合存储；Serverless 同步；多语言契约生成
风险点 (已发生)	动态版本区间导致构建不可复现 → protobuf 启动失败
风险点 (未发生)	AWS SDK v1/v2 长期并存的维护成本累积
非风险点	Spring Session + Redis Cluster 会话外置
质量属性战术 (可用性)	故障隔离（部署单元拆分、Lambda 按域拆分）
质量属性战术 (性能)	减少存储开销（读写分离）、协议优化（gRPC/HTTP2）
质量属性战术 (可修改性)	限制依赖（版本 force 锁定）、抽象（ <code>IGrpcService</code> 接口）
质量属性战术 (可测试性)	本地云模拟（LocalStack）、接口分离（ <code>IGrpcService</code> ）

五、可用难点素材 (按适配度排序)

- 1. ② **protobuf 版本冲突** ★★ ★ —— 风险点 → 实际故障，论点完整
- 2. ⑤ **混合存储选型** ★★ ★ —— 最典型的权衡点
- 3. ⑥ **Serverless 同步** ★★ ★ —— 性能与一致性的冲突处理
- 4. ③ **AWS SDK v1/v2 并存** ★★ —— 可修改性的长期成本

六、踩坑提醒

坑	后果	正确做法
不提 ATAM 或提了不会用	明显不切题	背熟四阶段 + 四概念（敏感点/权衡点/风险点/非风险点）
术语用错	反而扣分	敏感点=关键性质；权衡点=影响多属性的敏感点；风险点=可能致问题的决策
只写「我们评估了」不写过程	空洞	按「场景收集 → 视图描述 → 属性分析 → 折中决策」四步写
不写量化目标	关键失分	质量属性场景必须有 响应度量
说「所有质量属性都达到最优」	逻辑错误	质量属性天然相互制约，必须写权衡
编造评估工具	高风险	本项目未见 Archimate 等工具使用证据；写「效用树 + 会议评审」是可行的
忘记写评估的产出	结构缺失	评估必须落到「识别了哪些点 + 形成了什么决策 + 有何效果」
与项目脱节，纯讲理论	判定为背书	每个理论点都要挂一个本项目实例

💡 **本主题的备考捷径：** 把上面「四、术语对照表」背下来。这张表把教材术语与你的项目一一对应，考场上只要题目沾「评估/质量属性/权衡」，就可以直接展开。这是本模板集中**理论性最强也最容易拿分**的一篇。

配套： 01- 微服务架构设计.md · 03- 数据架构与混合存储.md · 09- 架构演化与遗留系统改造.md